

ACTIVIDAD 2

Desarrollo de producto MVP con TDD y BDD

E Tickets Clásico Nacional

Municipal vs Comunicaciones



VS



DATOS DE ENTREGA	
Estudiante	YONI ALEXSANDER QUIRAN QUIRAN
Asignatura	Ingeniería del Software Avanzada
Entorno	Python y pytest en Anaconda para macOS
Fecha	4 de septiembre de 2026

Índice de contenidos

Apartado	Página
1 Introducción	2
2 Desarrollo con TDD y BDD	3
3 Pruebas unitarias	4
4 Pruebas de aceptación	5
5 Evidencia final	6
6 Manual de usuario	7
7 Código fuente y GitHub	9
8 Conclusión y referencias	9

1 Introducción

E-Tickets es un MVP para la venta de entradas del partido Municipal vs. Comunicaciones. La versión actual conserva la interfaz visual de selección de localidades y añade la estructura obligatoria del proyecto base: Evento, SistemaVentas y VentaError.

1.1 Funcionalidades implementadas

- Registrar eventos con nombre, aforo y precio por entrada.
- Consultar entradas vendidas y disponibles.
- Vender entradas sin superar el aforo.
- Rechazar cantidades inválidas y eventos inexistentes.
- Calcular totales con descuentos entre 0 y 100 por ciento.
- Seleccionar localidades y asientos en el prototipo visual.

2 Desarrollo con TDD y BDD

Se trabajó sobre el repositorio base entregado por la docente. El archivo `tests/test_aceptacion.py` se mantuvo sin cambios. Primero se ejecutaron sus siete pruebas y fallaron porque los métodos contenían `NotImplementedError`. Después se escribieron pruebas unitarias pequeñas y se implementó únicamente el comportamiento necesario para hacerlas pasar.

2.1 Ciclo aplicado

Etapas	Trabajo realizado	Evidencia
Rojo	Ejecución del código base sin implementar.	7 pruebas fallidas
Verde	Implementación gradual de Evento y SistemaVentas.	21 pruebas superadas
Refactor	Validaciones y métodos auxiliares para evitar duplicación.	21 pruebas se mantienen

2.2 Estructura del proyecto

- `venta_entradas/modelos.py`: clases y reglas solicitadas.
- `tests/test_aceptacion.py`: siete pruebas oficiales sin modificar.
- `tests/test_unitarias.py`: catorce casos unitarios propios.
- `E_Tickets_Rojos_vs_Cremas.ipynb`: prototipo visual y ejecución de `pytest`.
- evidencias: resultados de las fases roja y verde.

3 Pruebas unitarias

Las pruebas unitarias verifican cada parte implementada antes de revisar el comportamiento completo. Los casos parametrizados para cantidades y descuentos se cuentan por separado; por eso se ejecutan 14 casos.

Grupo	Comportamientos comprobados	Casos
Evento	Constructor, entradas disponibles y disponibilidad según aforo.	3
Registro	Registro correcto y rechazo de nombres repetidos.	2
Ventas	Venta correcta, exceso, cero, negativo, inexistente y estado acumulado.	6
Descuentos	Cálculo válido, porcentaje negativo y porcentaje mayor que 100.	3

3.1 Correspondencia con los comentarios recibidos

Observación	Corrección realizada
Registrar evento	test_registrar_evento
Venta exitosa	test_vender_entradas_exito
Exceso de entradas	test_vender_mas_entradas_de_las_disponibles
Cantidad cero o negativa	test_vender_cero_o_menos_entradas
Descuento válido e inválido	test_descuento_por_cantidad y test_descuento_invalido
Persistencia del estado	test_estado_de_ventas_se_conserva_entre_compras

4 Pruebas de aceptación

Las siete pruebas de aceptación fueron proporcionadas por la docente y no se modificaron. Verifican el sistema desde el resultado que espera el usuario.

Prueba oficial	Resultado esperado
Crear evento	Inicia con cero ventas y aforo completo.
Vender dentro del aforo	Devuelve el total y actualiza la disponibilidad.
Superar el aforo	Lanza VentaError.
Evento inexistente	Lanza VentaError.
Cantidad inválida	Rechaza cero o menos entradas.
Total con descuento	Dos entradas de Q100 con 10 % dan Q180.
Descuento inválido	Rechaza porcentajes fuera de 0 a 100.

FASE ROJA · CÓDIGO BASE

```
$ python -m pytest tests/test_aceptacion.py -v
```

```
collected 7 items
```

```

crear_evento_inicia_sin_entradas_vendidas FAILED
vender_entradas_dentro_del_aforo              FAILED
no_vender_mas_entradas_que_el_aforo           FAILED
no_vender_evento_inexistente                  FAILED
no_vender_cantidad_invalida                   FAILED
calcula_total_con_descuento                   FAILED
descuento_invalido_lanza_error                 FAILED
7 failed in 0.03s · causa: NotImplementedError

```

```
7 failed in 0.03s · causa: NotImplementedError
```

Figura 1. Fase roja del código de partida: 7 pruebas fallidas.

5 Evidencia final

La ejecución completa se realizó con Python 3.13 de Anaconda y pytest 8.4.2. El resultado final fue 21 passed: 7 pruebas de aceptación y 14 casos unitarios.

```
FASE VERDE · PROYECTO FINAL

$ python -m pytest -v

collected 21 items

tests/test_aceptacion.py              7 PASSED
Evento: constructor y entradas disponibles PASSED
registrar_evento                      PASSED
vender_entradas_exito                 PASSED
vender_cero_o_menos_entradas          PASSED
vender_mas_entradas_de_las_disponibles PASSED
descuento_por_cantidad e invalido     PASSED
estado_de_ventas_se_conserva          PASSED
21 passed in 0.03s
```

21 passed in 0.03s

Figura 2. Fase verde: todas las pruebas oficiales y propias pasan.

5.1 Comandos de comprobación

```
python -m pytest tests/test_aceptacion.py -v
```

```
python -m pytest tests/test_unitarias.py -v
```

```
python -m pytest -v
```

6 Manual de usuario

6.1 Ejecutar las pruebas en Anaconda

1. Abrir Terminal desde macOS o Anaconda.
2. Entrar a la carpeta principal del proyecto.
3. Ejecutar `python -m pip install -r requirements.txt`.
4. Ejecutar `python -m pytest -v` y comprobar 21 passed.

6.2 Ejecutar el prototipo en Jupyter

1. Abrir `E_Tickets_Rojos_vs_Cremas.ipynb` en Jupyter.
2. Ejecutar la celda de la aplicación y abrir `http://127.0.0.1:8765`.
3. Elegir localidades y asientos y completar el formulario.
4. Ejecutar la celda de `pytest` para ver las 21 pruebas.
5. Ejecutar la última celda para detener la aplicación.

Selecciona tus asientos

Hasta 6 boletos por compra.

CANCHA

Palco

desde Q 550.00

P1-1

P1-2

P1-3

P1-4

P2-1

P2-2

P2-3

P2-4

Tribuna

desde Q 350.00

T1-1

T1-2

T1-3

T1-4

T1-5

T1-6

T2-1

T2-2

T2-3

T2-4

T2-5

T2-6

Preferencia

desde Q 250.00

F1-1

F1-2

F1-3

F1-4

F1-5

F1-6

F1-7

F2-1

F2-2

F2-3

F2-4

F2-5

F2-6

F2-7

Tu selección

P1-2 · Palco	Q 550.00
P1-3 · Palco	Q 550.00
Total	Q 1,100.00

Nombre del comprador

Nombre y apellido

Tarjeta de prueba

4242 4242 4242 4242

Vencimiento

12/30

CVV

123

Confirmar compra

Figura 3. Selección de localidades y asientos en E-Tickets.

6.3 Resultado de la compra

Después de confirmar la selección, la aplicación muestra el evento, el comprador, los asientos, el total y un código único del boleto.



Figura 4. Boleto digital generado por el prototipo.

6.4 Datos para la demostración

Campo	Valor
Tarjeta	4242 4242 4242 4242
Vencimiento	12/30
CVV	123
Límite visual	Hasta 6 boletos

7 Código fuente y GitHub

El repositorio público contiene el código solicitado, las pruebas oficiales sin modificar, las pruebas unitarias, la evidencia roja y verde y el notebook.

Repositorio público: <https://github.com/timpaniquiran-netizen/actividad-2-venta-entradas-tdd>

8 Conclusión

Con esta corrección pude adaptar mi plataforma al proyecto base entregado por la docente. Las pruebas me permitieron avanzar paso a paso y comprobar casos que antes no había incluido, como cantidades inválidas, descuentos y control del aforo. El sitio conserva su diseño original y ahora el código cumple con las siete pruebas de aceptación y con las pruebas unitarias propias.

8.1 Herramientas utilizadas

Python para la lógica, pytest para las pruebas, Jupyter y Anaconda para la ejecución, y GitHub para publicar el código.

8.2 Referencias

- Proyecto base: <https://github.com/albasagas/venta-entradas-tdd>
- pytest: <https://docs.pytest.org/>
- Jupyter: <https://docs.jupyter.org/>
- Python: <https://docs.python.org/3/>